

(T1) Tech Exercise Preparation

Workout/Exercise Logger Web Application

Aleksey Vorobyev

October 05, 2025

1. Description of the Web Application

For my project, I'm going to build a Workout/Exercise Logger web application. The idea is to help users plan their workouts, track their exercise routines, and keep an eye on their fitness progress.

The main functionality will include adding, viewing, editing, marking as complete, and deleting workout entries. Each workout entry will have these attributes:

- Exercise Name: A short description of the workout (like "Morning Run," "Chest Day," or "Yoga Session").
- Description: More details about the workout - things like distance, reps, sets, duration, or any notes I want to remember (this one's optional).
- Due Date: When I'm planning to do the workout.
- Status: Where the workout is at right now - either "Planned," "In Progress," or "Completed."
- Priority: How important the workout is for my fitness goals - "Low," "Medium," or "High."

My Workout/Exercise Logger will let users:

- See all their workouts in a list.
- Add new workouts to their schedule.
- Edit workout details like the name, description, due date, and status.
- Mark workouts as completed when they're done.
- Delete workouts they don't need anymore.
- Filter workouts by status or priority.

2. Links to Relevant Tutorials

To help me get started with building this application, I found some tutorials that cover the basics of Spring Boot, React, SQLite, and GitHub:

- Tutorial 1: Spring Boot Official Getting Started Guide
 - (Related links should be included here.)
 - This is the official Spring Boot tutorial. It shows how to create a basic Spring Boot application from scratch, including setting up the project and creating REST APIs. I'll need this for building the backend.
- Tutorial 2: Building a REST API with Spring Boot
 - (Related links should be included here.)
 - This one teaches how to build RESTful web services using Spring Boot. It covers the HTTP methods I'll need like GET, POST, PUT, and DELETE for the CRUD operations in my workout logger.
- Tutorial 3: React Official Tutorial: Getting Started
 - (Related links should be included here.)
 - The official React docs and tutorial. It covers the basics of React like components, state, and hooks. I need this to build the frontend part of my app.
- Tutorial 4: Connecting Spring Boot with SQLite Database
 - (Related links should be included here.)
 - This tutorial explains how to set up Spring Boot to work with SQLite. It covers JPA configuration and how to map entities to the database, which I'll use to store all the workout data.
- Tutorial 5: Getting Started with Git and GitHub
 - (Related links should be included here.)
 - This covers the basics of using Git and GitHub for version control. I'll use this to track my code changes and keep everything backed up in the cloud.
- Tutorial 6: Connecting React Frontend with Spring Boot Backend
 - (Related links should be included here.)
 - This shows how to connect a React frontend to a Spring Boot backend API. It covers important stuff like CORS configuration and making API calls from React to the backend.

3. Web Framework, Database, and Version Control System

For my web application, I'm planning to use the following technologies:

- Web Framework:
 - Spring Boot (Backend): I chose Spring Boot for the backend because it's pretty straightforward to set up and has a lot of built-in features. It'll handle all the server-side stuff like processing requests, managing the business logic, and talking to the database. Plus it makes creating REST APIs really easy.
 - React (Frontend): For the frontend, I'm using React because it makes building interactive user interfaces easier. I can create reusable components and manage

the app's state without too much hassle. React will handle displaying the workouts and sending requests to the Spring Boot backend.

- Database:
 - SQLite: I went with SQLite because it's simple to set up and doesn't need a separate database server running. Everything is stored in one file, which makes it perfect for this kind of project. Spring Boot works well with SQLite through JPA and Hibernate, so I can interact with the database using Java objects instead of writing a bunch of SQL queries.
- Version Control System:
 - GitHub: I'll be using GitHub to keep track of my code and any changes I make. It's also a good backup in case something goes wrong. I might organize it as one repository with separate folders for the backend and frontend, or possibly two separate repos depending on how it goes.

The way this will work is that React (the frontend) will talk to Spring Boot (the backend) using REST API calls. Spring Boot will then handle all the database operations with SQLite to save and retrieve workout information.

4. Cloud Hosting Service

For hosting my web application, here's what I'm planning:

- Cloud Hosting:
 - Google Cloud Platform (GCP): I'm going to use Google Cloud's E2 instance to host my application. It's pretty affordable and should have enough resources to run both the Spring Boot backend and React frontend. Google Cloud seems reliable and has good documentation.
- Operating System:
 - Ubuntu: The E2 instance will be running Ubuntu Linux. I picked Ubuntu because it's stable, widely used, and there's tons of community support if I run into issues. Right now I'm developing everything locally on my computer, but I'll deploy it to the Google Cloud E2 instance when it's ready.
- Deployment Strategy:
 - I'll package the Spring Boot backend as a JAR file and run it on the E2 instance.
 - For the React frontend, I'll build it for production and serve it from the same server.
 - I'll also need to set up the firewall rules on Google Cloud to allow web traffic to reach the application.

5. Development Environment

Here are the tools I'll be using for development:

- IDE:
 - Visual Studio Code (VSCode): I'm using VSCode for both the Spring Boot backend and the React frontend. It's free, lightweight, and has extensions for pretty much everything I need. For Spring Boot, I'll install the Extension Pack for Java and the Spring Boot Extension Pack. For React, I'll use extensions like ESLint, Prettier, and some React code snippets. VSCode also has built-in Git support which is convenient.
- Java & Spring Boot Setup:
 - I'll need to install Java JDK (version 17 or higher) on my computer.
 - I'll use Spring Initializr to generate the initial project with all the dependencies I need like Spring Web, Spring Data JPA, and the SQLite driver.
 - Maven will handle the build process and dependency management.
- Node.js & React Setup:
 - I need to install Node.js (version 18 or higher) and npm for managing JavaScript packages.
 - I'll use Create React App or Vite to set up the React project quickly.
 - npm will manage all the JavaScript dependencies and run the development server.
- Additional Tools:
 - Postman: I'll use this to test my REST API endpoints while I'm building them.
 - Git: For version control, integrated with VSCode.
 - Browser Developer Tools: For debugging the React frontend and checking network requests.

Both Spring Boot and React come with development servers, so I can test everything locally before deploying to Google Cloud.